

Informe Final — Implementación de Tetris en C++ mediante Manipulación de Bits

Oscar Trujillo

Facultad de Ingeniería, Universidad de Antioquia

Informática II — Semestre 2026-1

Profesor: Aníbal Guerra

Marzo 2026

Repositorio: <https://github.com/Ostras21/tetris-en-c->

Resumen

El informe presenta el análisis, diseño y planificación de una implementación del juego Tetris en lenguaje C++ bajo el entorno Qt haciendo uso de operaciones a nivel de bits y gestión explícita de memoria dinámica. Se describen las decisiones de representación de datos tomadas con base en criterios de eficiencia de memoria, la arquitectura modular del sistema y el plan de implementación propuesto para las entregas establecidas en el Desafío I de Informática II.

Palabras clave: Tetris, C++, operadores a nivel de bits, memoria dinámica, programación modular, Qt Creator.

1. Contextualización

El proyecto consiste en la implementación de una versión simplificada del juego Tetris solamente usando la interfaz de consola. A diferencia del juego original, esta versión funciona por turnos, el sistema muestra el estado del tablero, solicita una acción al usuario y procesa el resultado.

El desarrollo se limita en las restricciones propias de la guía del desafío, se prohíbe el uso de la STL, de structs, de strings y de cualquier instrucción de ANSI C. Toda la lógica central debe apoyarse en operaciones a nivel de bits y en gestión explícita de memoria dinámica mediante punteros.

Estas restricciones son con el fin de hacer un programa eficiente, donde los recursos de memoria y procesamiento son limitados y cada decisión tiene un impacto directo en la eficiencia del sistema.

2. Análisis del Problema y Consideraciones de Solución

2.1 El tablero

El tablero es una cuadrícula de A columnas y L filas donde cada celda está ocupada o libre, el ancho debe ser múltiplo de 8 y tanto el ancho como el alto deben tener mínimo 8 bloques, la primera decisión importante fue cómo representar ese tablero en memoria. La opción más obvia sería un arreglo de booleanos o enteros, pero eso desperdicia mucha memoria. Si cada celda es solo un bit (ocupada o libre), lo más eficiente es usar exactamente eso, bits.

La solución fue representar cada fila como un arreglo de bytes (`uint8_t`), donde cada bit corresponde a una celda. Como el ancho siempre es múltiplo de 8, cada fila ocupa exactamente

ancho/8 bytes sin desperdiciar nada. El tablero completo es entonces un arreglo dinámico de punteros, donde cada puntero apunta a una fila. Esto permite manipular filas individualmente con solo mover punteros, lo cual es muy eficiente al momento de eliminar filas completas.

2.2 Las piezas

Hay 7 tipos de piezas, I, O, T, S, Z, J, L cada una con hasta 4 rotaciones, Una pieza en cualquier rotación cabe dentro de una cuadrícula de 4x4 celdas, lo que son 16 bits en total. Eso significa que cualquier forma de Tetris se puede representar en un único `uint16_t`, donde cada bit activo indica una celda ocupada de la pieza.

El requisito más exigente aquí fue que las rotaciones no se pueden almacenar, hay que calcularlas en tiempo real con operaciones de bits. La rotación de 90 grados en el sentido de las manecillas del reloj sobre una matriz de 4x4 tiene una transformación matemática clara, cada bit en la posición (fila, col) se mueve a la posición (col, 3-fila). Eso se puede implementar completamente con desplazamientos y máscaras de bits.

2.3 La lógica de turno

En cada turno el usuario puede mover la pieza a la izquierda a derecha, bajarla o rotarla. Antes de aplicar cualquier movimiento hay que verificar si es válido, los bits de la pieza en su nueva posición no deben superponerse con bits ya ocupados del tablero ni salirse de sus límites. Si el movimiento es válido se aplica, si no, simplemente se ignora.

2.4 Detección de colisiones

La colisión se detecta proyectando los bits de la pieza sobre las posiciones del tablero que ocuparía y aplicando el operador AND. Si el resultado es distinto de cero, hay al menos un bit en común entre la pieza y el tablero, lo que significa que hay una colisión. También se verifican los

límites del tablero, si algún bit activo de la pieza cae fuera del rango válido, el movimiento también se rechaza.

2.5 Limpieza de filas

Cuando una fila tiene todos sus bits en 1, está completa y debe eliminarse. Las filas superiores bajan una posición, entonces la forma más eficiente de hacer esto no es copiar bytes sino simplemente reasignar punteros, se libera la fila completa, se desplazan los punteros hacia abajo y se asigna una nueva fila vacía en la parte superior. Mover un puntero es una operación de pocos bytes sin importar cuán ancho sea el tablero.

2.6 Game Over

El juego termina cuando una pieza nueva no puede colocarse en su posición inicial, que es el centro de la parte superior del tablero. En ese momento se detecta colisión inmediata al generar la pieza y se activa la condición de Game Over.

3. Esquema de Tareas y Arquitectura del Sistema

Para organizar el desarrollo se dividió el sistema en cuatro módulos independientes, cada uno con una responsabilidad clara. Esta separación no solo es un requisito de la guía sino que facilita entender, probar y corregir cada parte por separado.

3.1 Módulo Tablero (board.h / board.cpp)

Este módulo maneja todo lo relacionado con el tablero, crea y libera su memoria dinámica, verificar si una pieza colisiona con el tablero o los bordes, fija una pieza al tablero cuando ya no puede bajar más, detectar y eliminar filas completas y consultar si una celda específica está ocupada.

3.2 Módulo Pieza (piece.h / piece.cpp)

Este módulo define el comportamiento de las piezas calcula la forma de cualquier pieza en cualquier rotación usando operaciones de bits, genera piezas nuevas de forma aleatoria y las ubica en el centro superior del tablero, y permite moverlas y rotarlas verificando siempre que el movimiento sea válido.

3.3 Módulo Juego (game.h / game.cpp)

Este módulo controla todo el flujo del juego, le pide al usuario las dimensiones del tablero al inicio, controla el ciclo de turnos, lee la entrada del usuario, coordina las acciones entre el tablero y la pieza, y detecta el Game Over.

3.4 Módulo Display (display.h / display.cpp)

Este módulo se encarga solode la visualización. Imprime el tablero en consola fila por fila, superponiendo la pieza activa sobre el estado fijo del tablero. Usa "#" para celdas ocupadas y "." para celdas vacías. También imprime el menú de acciones, la puntuación y el mensaje de Game Over.

3.5 Diagrama de dependencias

Hay una jerarquía, el main.cpp crea e inicia el objeto Juego. Juego coordina a Tablero, Pieza y Display. Ni Tablero ni Pieza conocen la existencia de Juego lo que mantiene el acoplamiento bajo y cada módulo independiente.

4. Algoritmos Implementados

4.1 Representación del tablero con bits

El tablero se representa como `uint8_t**` filas, donde cada `filas[i]` es un arreglo de ancho/8 bytes. Para saber si la celda x, y está ocupada se calcula el byte correspondiente con `x/8` y el bit

dentro de ese byte con $7-(x\%8)$, luego se aplica un desplazamiento y una máscara AND con 1. Para ocupar una celda se usa OR con el bit correspondiente.

4.2 Cálculo de rotaciones con bits

Cada pieza parte de una forma base codificada como `uint16_t`. Para calcular la rotación número r , se aplica r veces la transformación de 90 grados, cada bit en la posición fila, col de la cuadrícula 4x4 se mueve a la posición col, 3-fila. Esto se implementa con dos ciclos anidados que recorren la cuadrícula, extraen cada bit activo con un desplazamiento y lo insertan en su nueva posición con OR. No se almacena ninguna rotación precalculada.

4.3 Detección de colisiones

Para verificar si una pieza puede moverse a una nueva posición se recorre su cuadrícula 4x4 bit a bit. Por cada bit activo se calcula la posición real en el tablero sumando la posición de la pieza. Si esa posición está fuera del tablero o la celda correspondiente está ocupada, se retorna colisión inmediatamente.

4.4 Limpieza de filas completas

Se recorre el tablero de abajo hacia arriba. Para cada fila se verifica si todos sus bytes son todos los bits en 1, si es así, se libera esa fila con delete, se desplazan los punteros superiores hacia abajo reasignando `filas[j] = filas[j-1]`, y se asigna una nueva fila vacía en la posición 0. El recorrido de abajo hacia arriba evita saltarse filas que podrían quedar completas después de un desplazamiento.

4.5 Generación aleatoria de piezas

Al crear una nueva pieza se usa `rand() % 7` para seleccionar uno de los 7 tipos de forma uniforme, se inicia con `srand(time(0))` al inicio del juego para que la secuencia sea diferente en cada partida. La pieza aparece siempre en $\text{posX} = (\text{ancho}/2) - 2$ y $\text{posY} = 0$, lo que la centra en la parte superior del tablero.

5. Problemas de Desarrollo Encontrados

5.1 Restricción sobre las rotaciones

El primer problema importante fue entender cómo calcular las rotaciones sin almacenar nada. La solución inicial era guardar las 4 rotaciones de cada pieza como constantes, pero el profesor aclaró que eso no estaba permitido. Fue necesario pensar en la rotación como una transformación matemática sobre los bits, lo que llevó a implementar el algoritmo de transposición y reflexión sobre la cuadrícula 4x4.

5.2 Librerías permitidas

Tenía dudas sobre qué librerías estaban autorizadas. Se confirmó que el random sí podía usarse lo que habilitó `cstdlib` para `rand()` y `srand()`. Para la salida en consola se optó por `QTextstream` de Qt en lugar de `cout` o `printf`, ya que `cout` requiere la STL y `printf` es ANSI C, ambas prohibidas.

5.3 Errores de compilación al iniciar

Al abrir el proyecto en Qt Creator por primera vez aparecían errores de tipos no reconocidos entre módulos. El problema era que Qt Creator no había indexado bien los archivos recién creados. Se resolvió cerrando y volviendo a abrir el proyecto, y verificando que todos los `includes` entre archivos `.h` estuvieran correctamente definidos. También muchos problemas a la hora de hacer toda la arquitectura del programa.

5.4 Configuración del proyecto Qt

El archivo `tetris.pro` inicialmente no incluía el módulo core de Qt, lo que causaba que `QTextStream` no fuera reconocido. Se corrigió agregando `QT += core` al archivo de configuración.

6. Evolución de la Solución

El desarrollo siguió un orden lógico dictado por las dependencias entre módulos, primero se definió la arquitectura completa en los archivos .h, lo que obligó a pensar con claridad qué responsabilidad tiene cada clase antes de escribir una sola línea de implementación.

Luego se implementó board.cpp como primer módulo, ya que es la base sobre la que todo lo demás opera. Una vez que el tablero funcionaba correctamente con sus operaciones de bits, se implementó piece.cpp con el algoritmo de rotación. Game.cpp fue el siguiente paso natural al tener los dos componentes principales listos, y finalmente display.cpp completó el sistema con la visualización.

Referencias

Stroustrup, B. (2013). The C++ programming language (4.a ed.). Addison-Wesley.

Tetris (2025). En Wikipedia. <https://en.wikipedia.org/wiki/Tetris>